

Person tracking from videos

Name: Su YANG

Student ID: 20417464

Module code: COMP3065

Coursework number: 1

Date: 2025.5.1

1. System Overview

In this research project, Python was selected as the primary programming language due to its excellent interpretability and structured code, which enhance both research and development processes. The object tracking task was methodically segmented into several stages. The initial stage—object detection—utilized YOLOv5 for implementation. Following the detection outcomes, a Kalman filter was employed to execute object tracking and visualize motion trajectories effectively. Finally, PyQt5 was utilized to construct the graphical user interface (GUI) and seamlessly integrate the entire system. The subsequent sections will provide experimental results along with comprehensive explanations for each of the five modules constituting the system.

2. Project Objectives and Design Motivation

This project seeks to develop a multi-object pedestrian detection and tracking system employing deep learning techniques, designed to sustain elevated levels of detection accuracy and tracking stability in intricate environments characterized by challenges such as occlusion, variations in lighting, and rapid motion. By leveraging the real-time detection capabilities of YOLOv5 in conjunction with the temporal prediction functionality of the Kalman filter, the system not only achieves reliable target recognition but also mitigates trajectory loss during instances of detection failures, thereby enhancing its overall robustness. Additionally, the system incorporates a graphical user interface (GUI) to facilitate improved user interaction, rendering it appropriate for educational demonstrations as well as practical applications in surveillance deployment.

3. Object Detection Algorithms

3.1 Dataset Selection

The dataset utilized in this research is the INRIA Person Dataset, which comprises a collection of labeled images showcasing individuals in standing or walking positions. This dataset was compiled by Navneet Dalal for the purpose of advancing research on the detection of upright pedestrians in images and videos. The INRIA dataset includes 614 positive training images that collectively feature 1,237 pedestrians, alongside 1,218 negative training images. The test set contains 288 positive images with 589 pedestrians and 453 negative images. Most of the pedestrians in the photos are in upright poses with heights greater than 100 pixels. It should be noted that some annotations may be inaccurate. A sample of the dataset is shown in Figure 3-1.



Figure 3-1 INRIA Dataset

3.2 Algorithm Selection

In contemporary object detection, most methods are based on deep learning, which typically yields high accuracy and robust performance. In this subsection, we compare the performance of a traditional object detection algorithm with that of a deep learning-based method on the same INRIA dataset. The conventional method employed combines Histogram of Oriented Gradients (HOG) and Support Vector Machine (SVM), while the deep learning approach utilizes the YOLOv5 model.

3.2.1 HOG and SVM Integration Mechanism

First, the image is divided into many small, connected regions, referred to as *cells*. For each cell, gradient magnitudes and orientations are computed for all pixels. A one-dimensional histogram of gradient directions is then generated for each cell. To enhance robustness to illumination and shadow variations, contrast normalization is applied to the histograms over larger spatial regions known as *blocks*. These normalized block descriptors are referred to as HOG descriptors.

The final feature vector is formed by concatenating the HOG descriptors of all blocks within the detection window. This feature vector is then fed into an SVM classifier to perform pedestrian detection. The dataset is split into positive and negative samples: positive samples contain images with pedestrians (see Figure 3-2), while negative samples are those without pedestrians (see Figure 3-3).

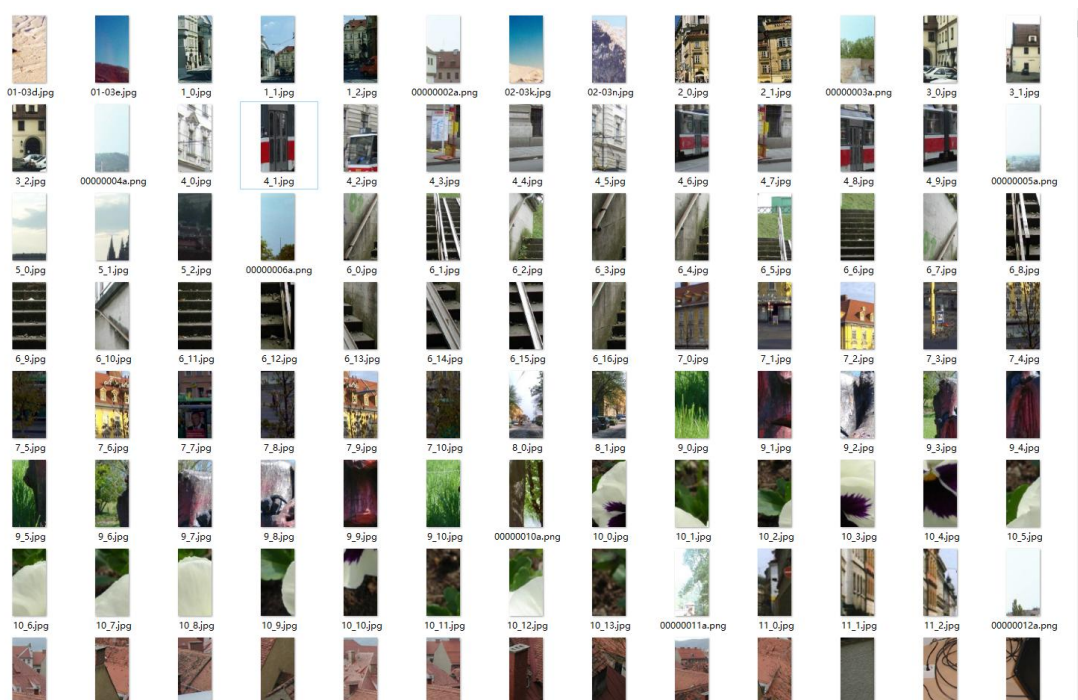


Figure 3-2 Illustration of Negative Samples

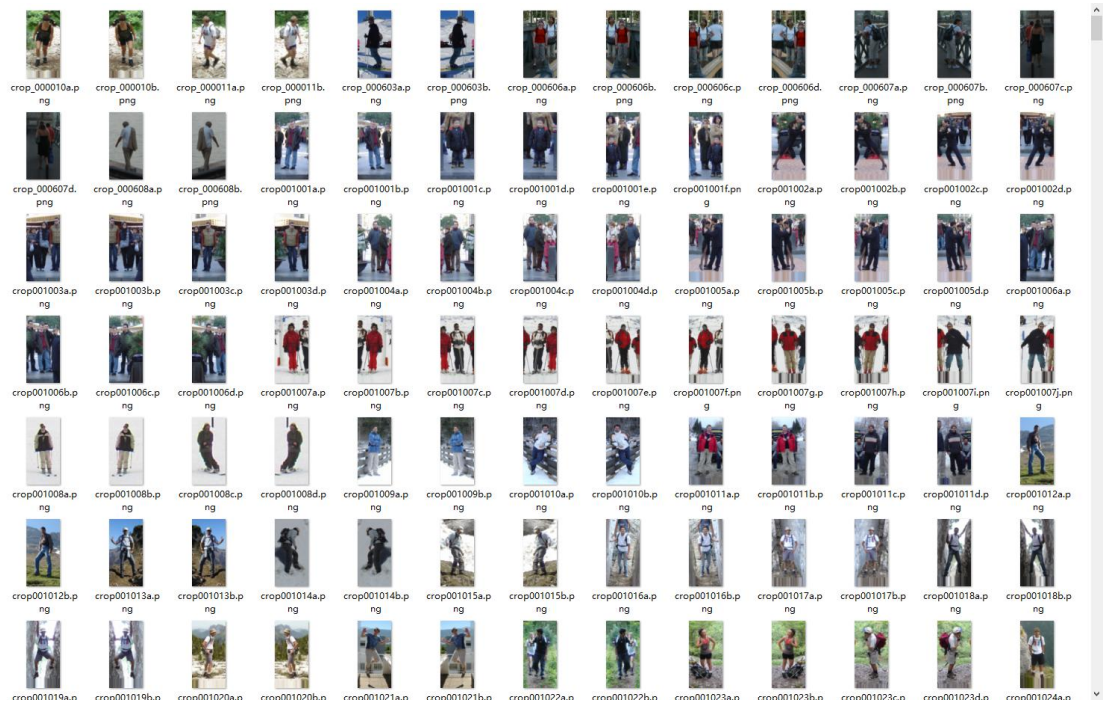


Figure 3-3 Illustration of Positive Samples

Prior to the application of Non-Maximum Suppression (NMS), the detection results are shown in Figure 3-4. After applying NMS, the improved results are presented in Figure 3-5. It can be observed that the pedestrian bounding boxes are more accurate after incorporating NMS. This improvement is due to NMS filtering out redundant detection boxes that fall below the defined confidence threshold.

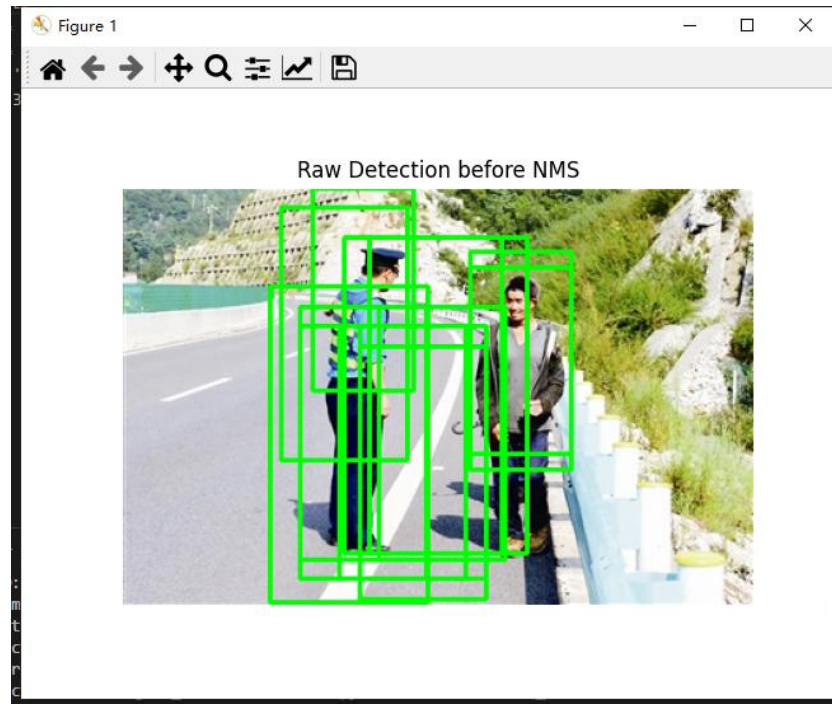


Figure 3-4 Before Applying Non-Maximum Suppression (NMS)

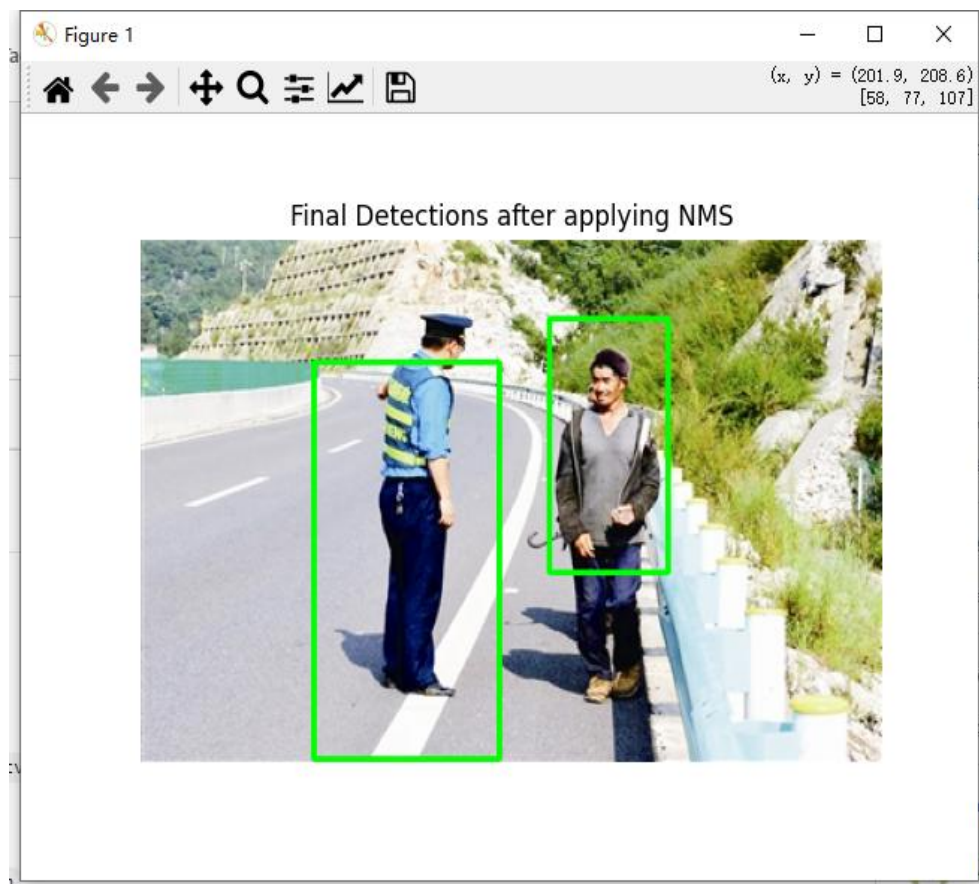


Figure 3-5 After Applying Non-Maximum Suppression (NMS)

3.2.2 YOLOv5

YOLOv5 (You Only Look Once version 5) is a deep learning-based object detection model widely used in computer vision. As the fifth iteration of the YOLO series, it offers multiple model scales—such as s, m, l, and x—to accommodate different hardware environments. Implemented in PyTorch, YOLOv5 features a comprehensive training and inference pipeline. Due to its real-time performance, ease of use, and open-source nature, YOLOv5 has gained popularity in various applications, including autonomous driving, security surveillance, and industrial inspection. The overall network architecture of YOLOv5 is illustrated in Figure 3-6.

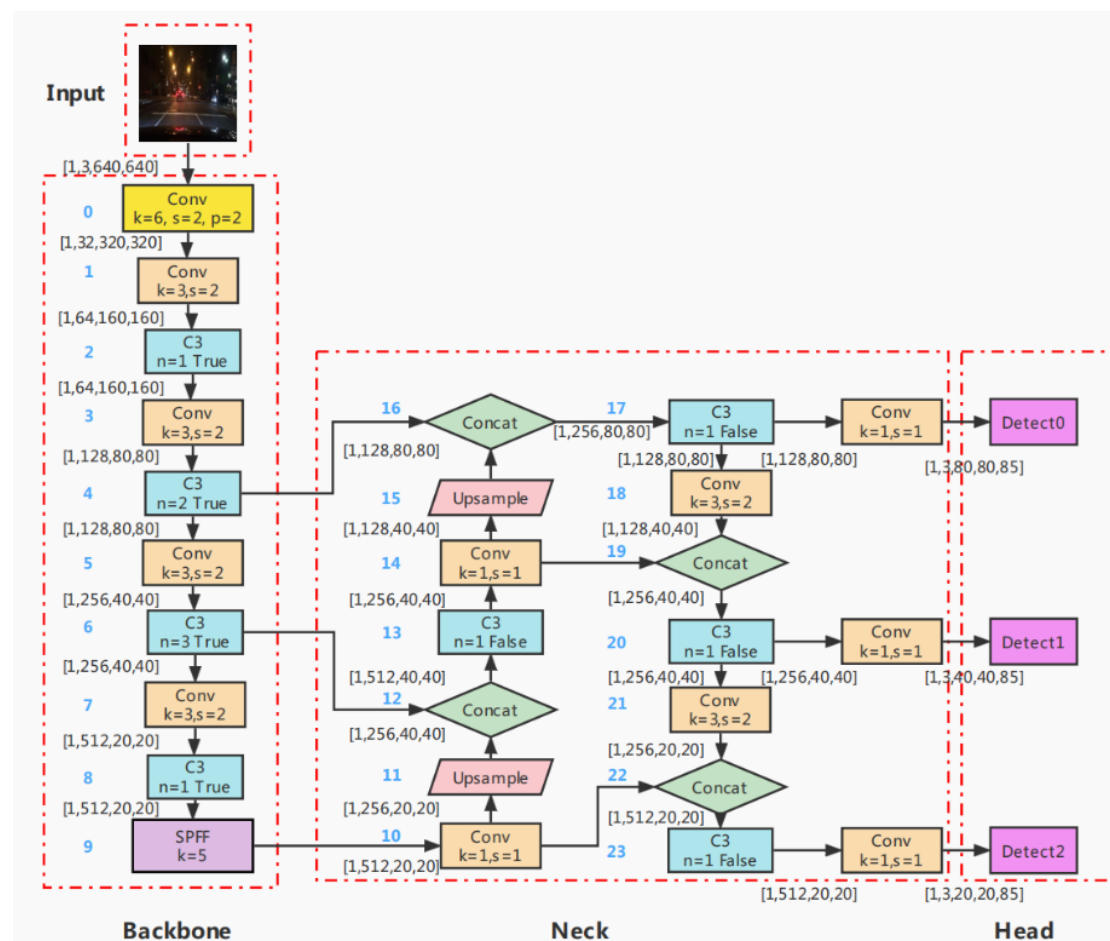


Figure 3-6 YOLOv5 Network Architecture Diagram

In this project, the YOLOv5s model was selected for its lightweight design and competitive performance. A key distinction from traditional object detection

approaches is in data usage: traditional methods typically rely on a binary division of positive and negative samples, whereas deep learning models like YOLOv5 require manually annotated data. An example of data annotated in YOLO format is shown in Figure 3-7.

```
0 0.6161567877629063 0.5270833333333333 0.16347992351816443 0.6020833333333333
0 0.9096558317399618 0.28541666666666665 0.044933078393881457 0.20625
0 0.9550669216061185 0.2703125 0.040152963671128104 0.16354166666666667
```

Figure 3-7 YOLO Format Annotation Example

The training results of the YOLOv5 network are shown in Figure 3-8. As depicted, the metrics such as mAP0.5, mAP0.5:0.95, precision, and recall all reached high levels of accuracy.

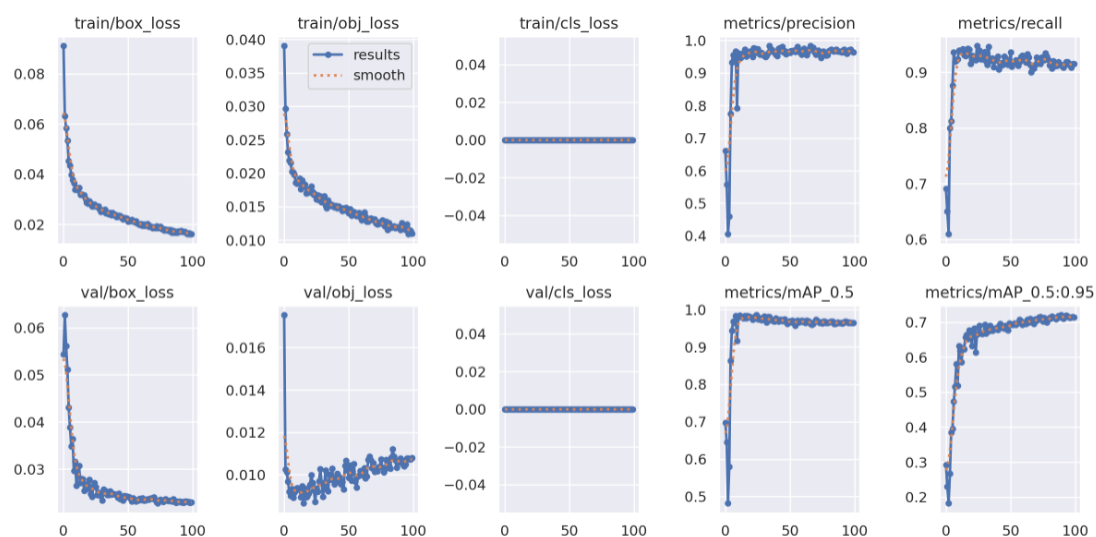


Figure 3-8 YOLOv5 Training Metrics Progression

The detection results on the validation set using the trained model are shown in Figure 3-9, illustrating a portion of the validation outcomes.



Figure 3-9 Validation Set Detection Results

Subsequently, the model was tested on the same images used in the traditional detection experiment, with results shown in Figure 3-10. The detection regions generated by YOLOv5 are clearly more accurate and complete, which explains why YOLOv5 was chosen over traditional detection algorithms in this study.



Figure 3-10 Detection Results on Traditional Algorithm Test Images Using

4. Object Tracking Algorithms

4.1 Introduction to the Kalman Filter

The Kalman Filter is a recursive prediction and filtering algorithm that simultaneously estimates current states and predicts future states of a system. It is defined by a set of recursive mathematical equations, offering an efficient computational means to estimate the state of a process while minimizing the mean squared error of the estimation. The Kalman Filter is widely used and highly powerful; it can estimate past, present, and even future states of a signal, even in the absence of a complete or accurate system model. It serves as a state estimator by fusing information from multiple sources, such as sensors, to improve overall accuracy.

As a small experiment, we simulated the trajectory of a moving ball and used the Kalman Filter to predict its future positions based on its previous position and system state. Assuming uniform motion (i.e., no abrupt changes in acceleration), the predicted trajectory aligns well with the true trajectory, as shown in Figure 4-1. This demonstrates one of the typical applications of the Kalman Filter. In our project, we leverage this predictive capability and integrate it with YOLOv5 to construct a complete object tracking system.

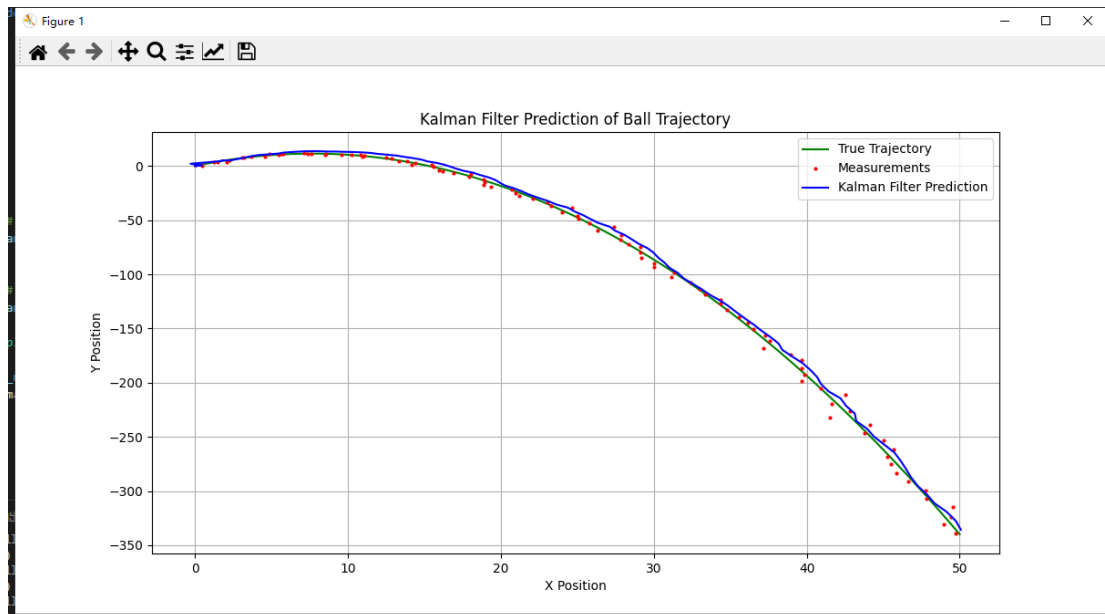


Figure 4-1 Predicted Trajectory Diagram

4.2 Integration with YOLOv5

From the previous subsection, we established that the Kalman Filter can effectively predict an object's trajectory on a two-dimensional plane. To integrate it with YOLOv5, modifications were made so that the detection results—specifically, the bounding box coordinates—are used as inputs to the Kalman Filter. These coordinates provide the object's position at the current frame. The Kalman Filter then predicts the position in the next frame, allowing continued tracking even when the object is temporarily occluded or missed by the detector.

This highlights the importance of accurate object detection as the foundation for successful object tracking. A robust detection algorithm such as YOLOv5 significantly reduces the probability of losing track of targets when fused with a predictive tracking algorithm like the Kalman Filter.

5. GUI Design and Implementation

The graphical user interface (GUI) was designed and implemented using PyQt5 with Qt Designer, as shown in Figure 5-1. The GUI preview is shown in Figure

5-2.

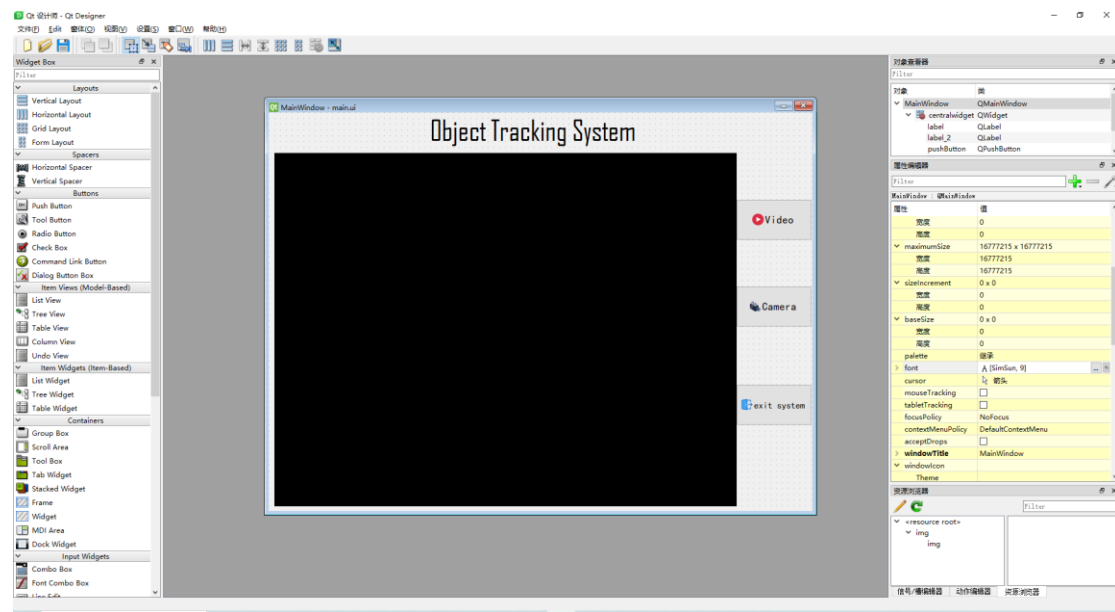


Figure 5-1 GUI Design Interface

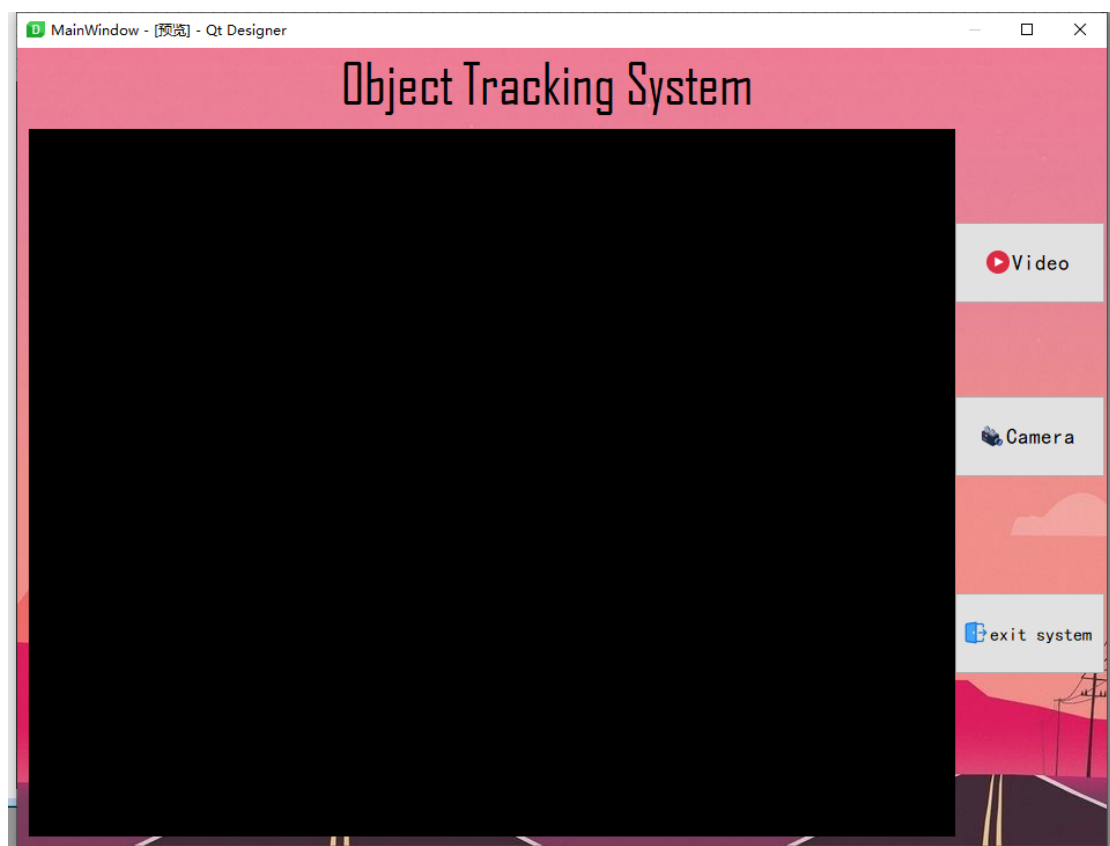


Figure 5-2 GUI Preview Interface

The interface primarily supports video input and live camera input. Static image

input is not supported, as object tracking inherently involves temporal information and motion analysis—features absent in still images. Therefore, image-based input was excluded from the design phase.

6. Tracking Results

Upon completing the GUI, the backend logic was implemented in Python, with sample code shown in Figure 6-1.

```
main.py > MainWindow > _init_
193     self.running = False
194
195
196 class MainWindow(QMainWindow):
197     def __init__(self):
198         super(MainWindow, self).__init__()
199         self.ui = Ui_MainWindow()
200         self.ui.setupUi(self)
201         self.setWindowTitle("Object Tracking System")
202         self.setFixedSize(self.width(), self.height())
203         self.setWindowFlags(self.windowFlags() & ~QtCore.Qt.WindowMaximizeButtonHint)
204
205         self.ui.pushButton.clicked.connect(self.handle_video_button)
206         self.ui.pushButton_2.clicked.connect(self.handle_camera_button)
207         self.ui.pushButton_3.clicked.connect(self.exit_system)
208
209         self.tracking_thread = None
210         self.current_mode = None # 'video' or 'camera'
211
212     def handle_video_button(self):
213         if self.tracking_thread and self.tracking_thread.isRunning():
214             self.stop_tracking()
215             self.ui.pushButton.setText("Video")
216             self.current_mode = None
217         else:
218             file_path, _ = QFileDialog.getOpenFileName(self, "Select video file", "", "Video files (*.mp4 *.avi *.mov)")
219             if file_path:
220                 self.start_tracking(file_path, False)
221                 self.ui.pushButton.setText("Stop")
222                 self.ui.pushButton_2.setEnabled(False)
223                 self.current_mode = 'video'
224
225     def handle_camera_button(self):
```

Figure 6-1 Partial Code Illustration

Three recorded videos were used as test inputs, and their respective results are illustrated in Figures 6-2, 6-3, and 6-4. In these visualizations, green bounding boxes represent YOLOv5 detection results, while red boxes denote Kalman Filter predictions. The trajectory of each object was drawn based on the Kalman predictions.

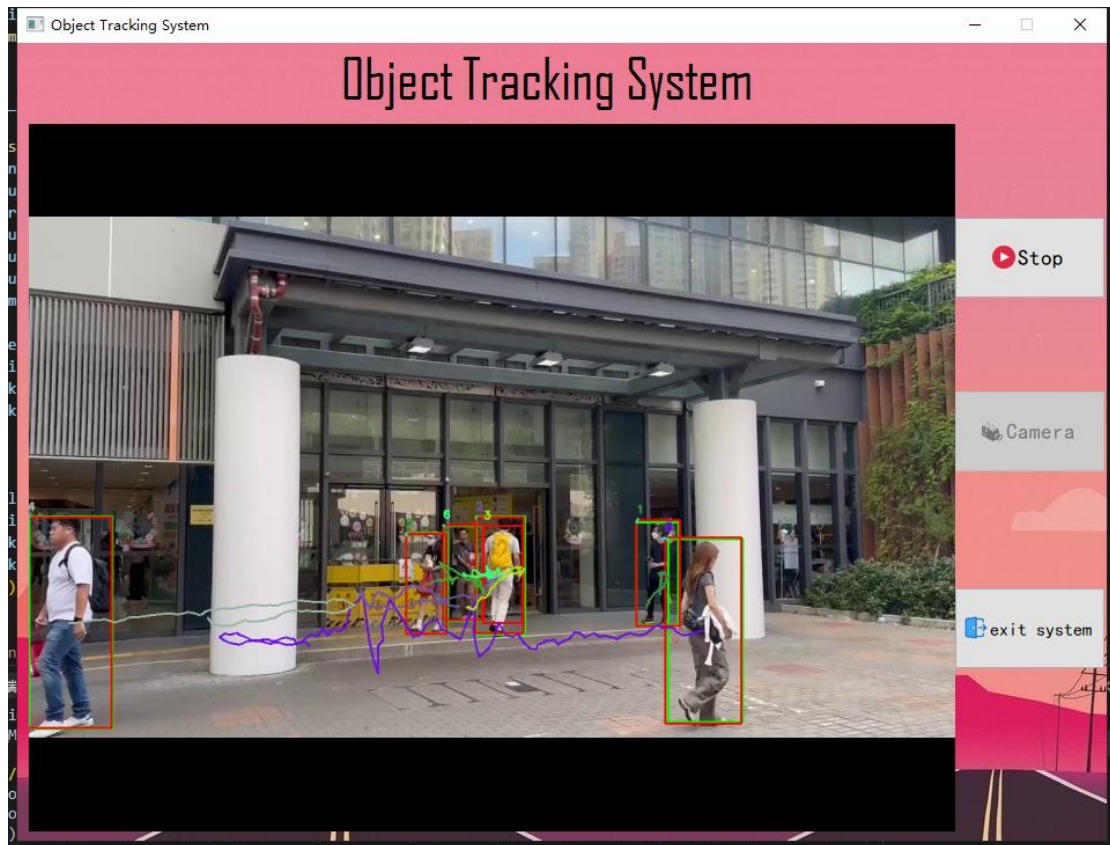


Figure 6-2 Tracking Result – Video 1

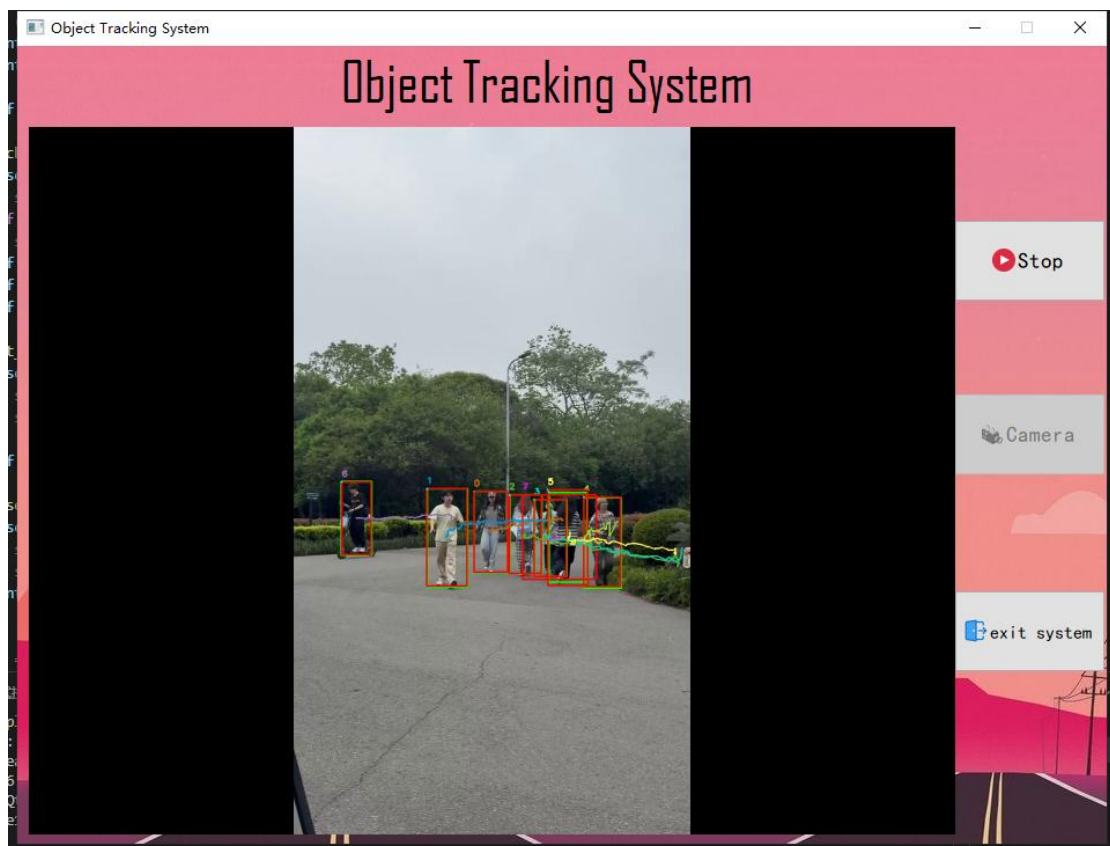


Figure 6-3 Tracking Result – Video 2

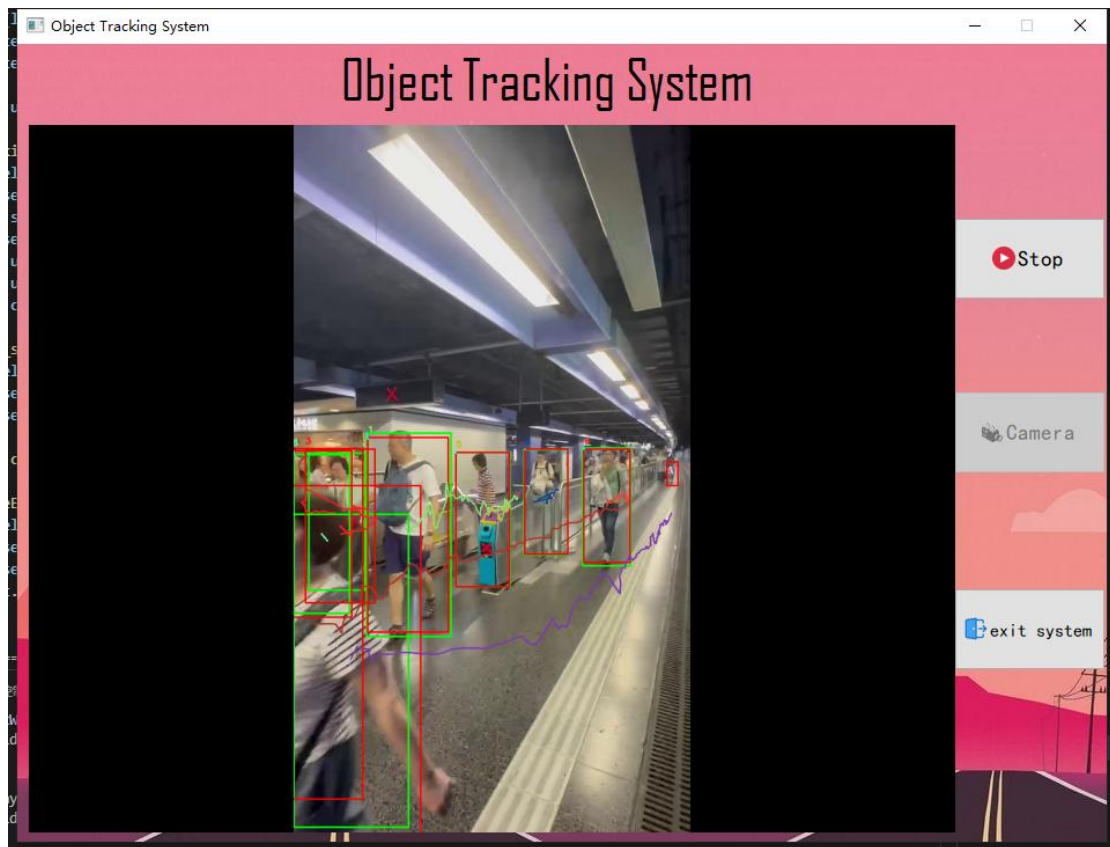


Figure 6-4 Tracking Result – Video 3

7. System Evaluation and Experimental Analysis

The system was tested on three self-recorded video clips, simulating scenarios of regular walking, multiple people with occlusions, and rapid movement. In cases of severe occlusion or fast motion, the system's ID stability decreased, resulting in multiple ID switches. This indicates that relying solely on position prediction through the Kalman Filter has certain limitations. In the future, incorporating a Re-ID mechanism (such as DeepSORT) could improve tracking consistency. Additionally, although YOLOv5s is lightweight, it still suffers from false detections under low-light conditions, suggesting a need for further optimization through data augmentation or the use of deeper models.

8. Summary

Through this experimental study, it became evident that object tracking is not an isolated task; it depends critically on accurate object detection as the first stage. The integration of YOLOv5 and the Kalman Filter achieved satisfactory tracking performance for most pedestrian targets, providing good visualization results. The system's strengths are mainly reflected in its clear structure, ease of expansion, real-time capability, and user-friendly interface. However, a few individuals were not properly detected or tracked. One limitation observed was that when a pedestrian disappeared from view, their tracking ID was reassigned upon re-entry. Objects with stable motion (i.e., consistent direction and speed) were more likely to be tracked successfully.

A notable drawback of the integrated model is its computational cost. When considering deployment on hardware devices, this level of processing demand may become a bottleneck.

To address the aforementioned shortcomings, the following improvement plans are proposed:

- Introduce advanced multi-object tracking algorithms such as DeepSORT or ByteTrack to achieve more robust ID assignment and tracking stability.
- Explore the use of new object detection models like YOLOv8 and RT-DETR to enhance the system's generalization capability.
- For hardware-constrained scenarios, further apply model compression and quantization to reduce deployment difficulty.
- Add more adjustable parameter options in the GUI to improve user convenience and configurability.

With these enhancements, it is believed that future versions of the system will achieve higher detection and tracking accuracy and be better suited for real-world application scenarios.